The generation of the reader instance method can be prevented by setting the option `:instance_reader` to `false`.

```
class A
  class_attribute :x, instance_reader: false
end

A.new.x = 1 # NoMethodError
```

For convenience `class_attribute` also defines an instance predicate which is the double negation of what the instance reader returns. In the examples above it would be called `x?`.

When `:instance_reader` is `false`, the instance predicate returns a `NoMethodError` just like the reader method.

If you do not want the instance predicate, pass `instance_predicate: false` and it will not be defined.

Defined in `active_support/core_ext/class/attribute.rb`

4.1.2 `cattr_reader`, `cattr_writer`, and `cattr_accessor`

The macros `cattr_reader`, `cattr_writer`, and `cattr_accessor` are analogous to their `attr_*`counterparts but for classes. They initialize a class variable to `nil` unless it already exists, and generate the corresponding class methods to access it:

```
class MysqlAdapter < AbstractAdapter
  # Generates class methods to access @@emulate_booleans.
  cattr_accessor :emulate_booleans
  self.emulate_booleans = true
end
```

Instance methods are created as well for convenience, they are just proxies to the class attribute. So, instances can change the class attribute, but cannot override it as it happens with `class_attribute`(see above). For example given

```
module ActionView
  class Base
    cattr_accessor :field_error_proc
    @@field_error_proc = Proc.new{ ... }
  end
end
```

we can access `field_error_proc` in views.

Also, you can pass a block to `cattr_*` to set up the attribute with a default value:

```
class MysqlAdapter < AbstractAdapter
  # Generates class methods to access @@emulate_booleans with default
value of true.
  cattr_accessor(:emulate_booleans) { true }
end
```

The generation of the reader instance method can be prevented by setting `:instance_reader` to `false` and the generation of the writer instance method can be prevented by setting `:instance_writer` to `false`. Generation of both methods can be prevented by setting `:instance_accessor` to `false`. In all cases, the value must be exactly `false` and not any false value.

```
module A
  class B
    # No first_name instance reader is generated.
    cattr_accessor :first_name, instance_reader: false
    # No last_name= instance writer is generated.
    cattr_accessor :last_name, instance_writer: false
    # No surname instance reader or surname= writer is generated.
    cattr_accessor :surname, instance_accessor: false
  end
end
```

A model may find it useful to set `:instance_accessor` to `false` as a way to prevent mass-assignment from setting the attribute.

Defined in `active_support/core_ext/module/attribute_accessors.rb`.